

Instructor: Date:

Learner: Regional Branch:

Program: Center:

CONTENT

- Let's Start! • Let's Explore! • Let's Practice! • Extension Activity • Target Words • Learning Points



Let's Start!

Sometimes looking at the details isn't what we need.



Learning Outcome:

Understand key information about the concepts, principles, and methodologies in simple spoken and written situations. Talk about programming techniques, use cases, kinds, and sequences in simple spoken and written situations.

Can Do Statements:

- I can describe what encapsulation is.
- I can explain different facts related to encapsulation.
- I can understand what encapsulation is in different contexts.

Ask your partner:

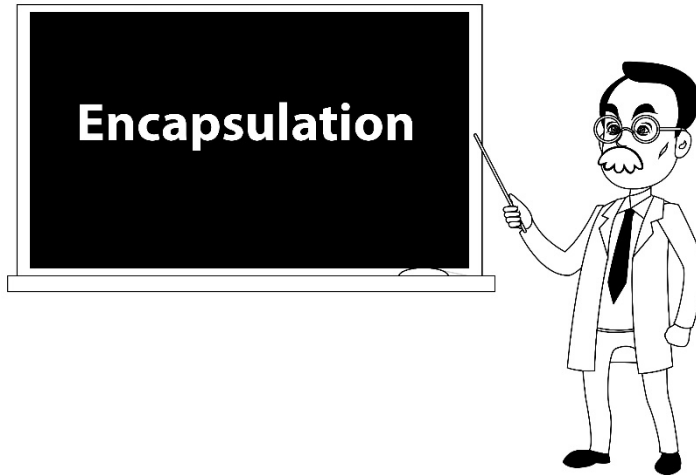
- What do you prefer, to create a program from scratch or to use a platform already created? Why?
- What are the benefits of focusing on the functionality of a program instead of the details that make it work?





Let's Explore!

A professor wants to explain how encapsulation works.

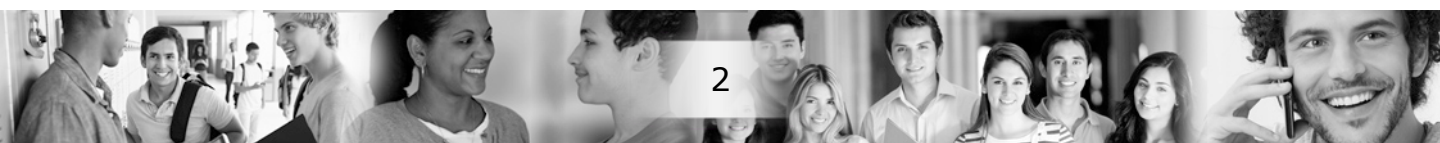


Instructions: Read the text below.

I want to start this lecture by explaining an important part of the Object-oriented Programming (OOP) called encapsulation. To do so, imagine when you take a taxi. Do you worry about the gas tank? Do you care about the status of the engine? Well, we don't care about those things, the only thing we worry about is our destination. That's what Encapsulation is about. The idea is to focus on the functionality instead of the details that make the program work; sometimes we have to keep it simple and show to the end user only what is fundamental to operate an application.

Nowadays, object-oriented programming makes things easy by joining everything in small "boxes" or classes, different from the past programming languages. Encapsulation allows us to enclose all those boxes, each one with different functions or subroutines, in order to hide details for users who don't need to know about them. Also, encapsulation protects the global variables and the structure of the program, and it keeps working without any modification from programmers or users.

Now, let's go back to the boxes I mentioned. These boxes contain both data structure and methods for manipulating the data. So, programmers identify these boxes and call them Classes. Some programmers admit working with object-oriented languages is sometimes confusing but they don't avoid it; instead, they find methods to make their life easier. To conclude, it is important to say that encapsulation helps us to organize our programming in a better way to customize an application and make it user-friendly. I know there is too much jargon we have to keep reviewing, but once you familiarize with all those terms, you will succeed in OOP.





Let's Practice!

What's encapsulation for?

Instructions: Select the correct answer.

1. Encapsulation ____ things easier for program users.
 - a. make
 - b. making
 - c. makes
 - d. to make
2. Sometimes programmers _____ need to see all the details.
 - a. does
 - b. doesn't
 - c. don't
 - d. to do
3. ____ encapsulation protect the structure of a program?
 - a. Do
 - b. Don't
 - c. Does
 - d. To do



Let's Practice!

Important Facts on OOP

Instructions: Read the sentences. Write the missing word(s) in the blank. Use: to enclose, using, and working.

1. Programs keep _____ without modifications from programmers or users.
2. Encapsulation allows _____ boxes.
3. Programmers admit _____ high-level classes create by experts.



Extension Activity

Let's talk about encapsulation.

Instructions: Complete the following steps. Fill in the Self-Assessment.

- Search more information about encapsulation.
- Write a list of 10 benefits of encapsulation for program development.
- Share the list with your class.

Self-Assessment

Yes No

- I list benefits of encapsulation.
- I interpret and explain encapsulation.
- I share my work with the class.

☐	☐
☐	☐
☐	☐



Target Words

- customize** (verb): to personalize or set up in such a way that satisfies the users' needs
e.g. Encapsulation helps us to organize our programming in a better way to customize an application and make it user-friendly.
- keep it simple** (expression): to do something in the easiest way possible
e.g. We have to keep it simple and show to the end user only what is fundamental to operate an application.
- jargon** (noun) language used in a specific context or area
e.g. I know there is too much jargon we have to keep reviewing.
- user-friendly** (adjective): easy to use
e.g. Encapsulation helps us to organize our programming in a better way to customize an application and make it user-friendly.
- avoid** (verb): to prevent something from happening
e.g. Some programmers admit working with object-oriented languages is sometimes confusing but they don't avoid it.
- end user** (noun): the person who is going to use a software
e.g. We have to keep it simple and show to the end user only what is fundamental to operate an application.



Learning Points

Simple Present Tense in All Forms

We use the **simple present tense** when we want to express regular situations and/or general facts.

Affirmative statements:

Subject + **verb** + complement

Example:

*I **design** high-quality classes.*

*She **designs** high-quality classes.*

Remember that when we use *he*, *she*, or *it* in an affirmative statement, it is necessary to add **-s**, **-es**, or **-ies** to the verb depending on its ending.

Examples:

*He **teaches** a class of XML at City University.*

*This Virtual object **runs** only if your PC **has** Java installed.*

*He always **tries** new methods to design programs.*

Negative statements:

Subject + **do not (don't) / does not (doesn't)** + **verb** + complement

Example:

*I **don't design** high-quality classes.*

*She **doesn't design** high-quality classes.*

Remember that when we use *he*, *she*, or *it* in negative statements, it not necessary to add **-s**, **-es**, or **-ies** to the verb because we have the auxiliary verb **doesn't**.

Examples:

*He **doesn't teach** a class of XML at City University.*

*This Virtual object **doesn't run** if your pc **doesn't have** Java installed.*

Questions:

Do / Does + Subject + **verb** + complement + ?

Example:

***Do** you **design** high-quality classes?*

***Does** she **design** high-quality classes?*





Learning Points

Remember that when we use *he*, *she*, or *it* in questions, it is not necessary to add **-s**, **-es**, or **-ies** to the verb as we have the auxiliary verb **does**.

Examples:

Does he **teach** a class of XML at City University?

Does this virtual object **run** on your PC?

Examples

- *Encapsulation allows us to enclose all those boxes, each one with different functions or subroutines.*
- *We don't care about those things.*
- *Do you worry about the gas tank?*

Infinitives vs. Gerunds

Gerunds are verbs in their -ing form. **Infinitives** are verbs followed by the preposition *to*.

Gerunds can be nouns if they are in the beginning of a sentence. You can recognize a **gerund** because it is a verb that ends in **-ing**.

Example: *I like studying new methods to develop programs.*

When we use **infinitives**, we separate two verbs with the preposition *to*.

Example: *I like to study new methods to develop programs.*

Sometimes we use **gerunds** or **infinitives** depending on the previous verb in the sentence, look at the next chart:

VERB + INFINITIVE	VERB + GERUND
Ask	Admit
Choose	Dislike
Decide	Imagine
Like	Practice
Propose	Spend



Learning Points

Example:

*I **admit using** platforms created by experts.*

*I **choose to generate** the codes from scratch.*

Gerunds and **infinitives** can change the meaning of a sentence after some verbs such as prefer, intend, start, continue, begin, and some others.

When they don't change the meaning	<i>They intend to develop a new program. They intend developing a new program.</i>
When they change the meaning	<i>I stop to check the details. I stop checking the details.</i>

Examples

- *Some programmers admit working with object-oriented languages is sometimes confusing, but they don't avoid it.*
- *It keeps working without any modification from programmers or users.*
- *I want to start this lecture by explaining an important part of the Object-Oriented Programming called encapsulation.*